Deep Learning: Optimization of Inference

Kiriukhov Grigorii

March 27, 2025

Contents

1 Calibration & Uncertainty Estimation

In this section, we will discuss classification tasks, including not only class assignment but also language modeling — that is, any scenario where neural networks are trained using a cross-entropy loss function.

$$\text{CrossEntropyLoss} = -\sum_{i=1}^{C} q_i \log p_i \ ,$$

where C are classes, q_i are true probabilities and p_i predicted probobilities.
Let us recall how the probabilities output by a neural network are formed. Typically, the network produces logits, which are then passed through a softmax function:

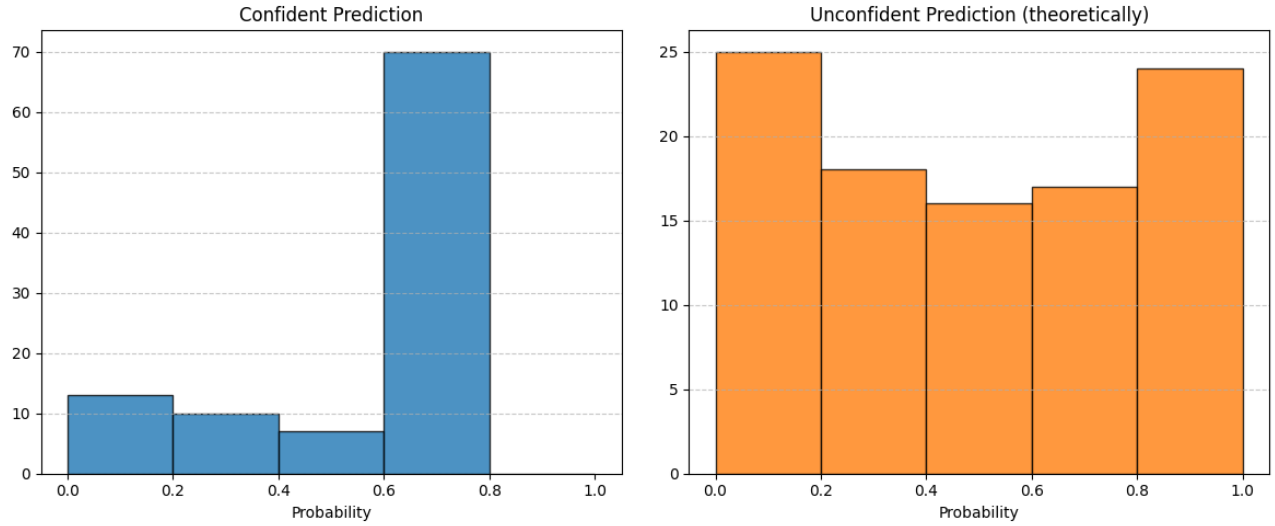
$$p_i = \frac{\exp(x_i)}{\sum_{k=1}^{c} \exp(x_i)} \ ,$$

where x_i are logits and p_i are probabilities.

To evaluate a model's confidence, it can be useful to analyze these probabilities. For example, in complex or ambiguous cases (e.g., medical prognosis), it is better to withhold predictions entirely rather than provide potentially unreliable answers.

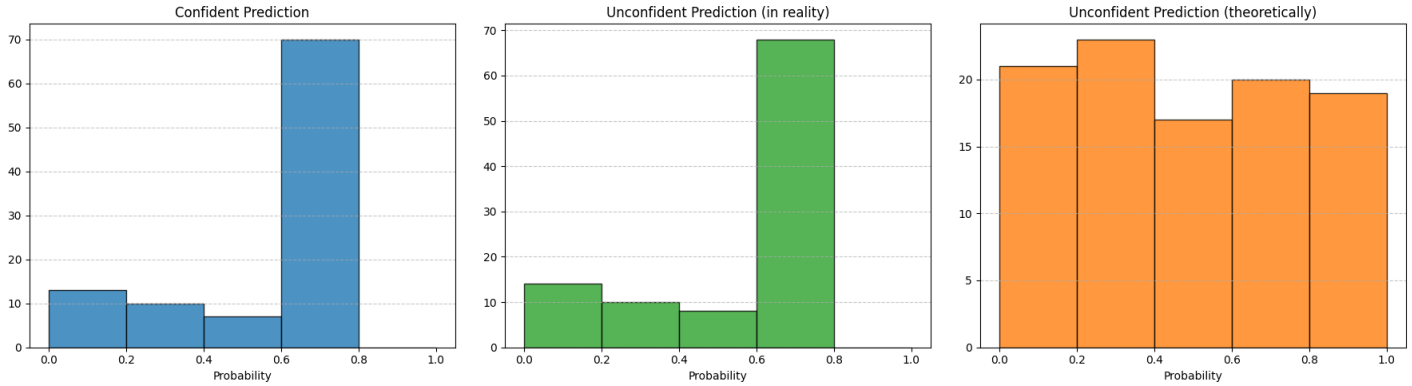
For practical applications of neural networks, it is critical to assess the uncertainty of predictions (i.e., how confident the model is). Since classifiers output probability distributions, let us use this idea.

We model the model's outputs as follows:



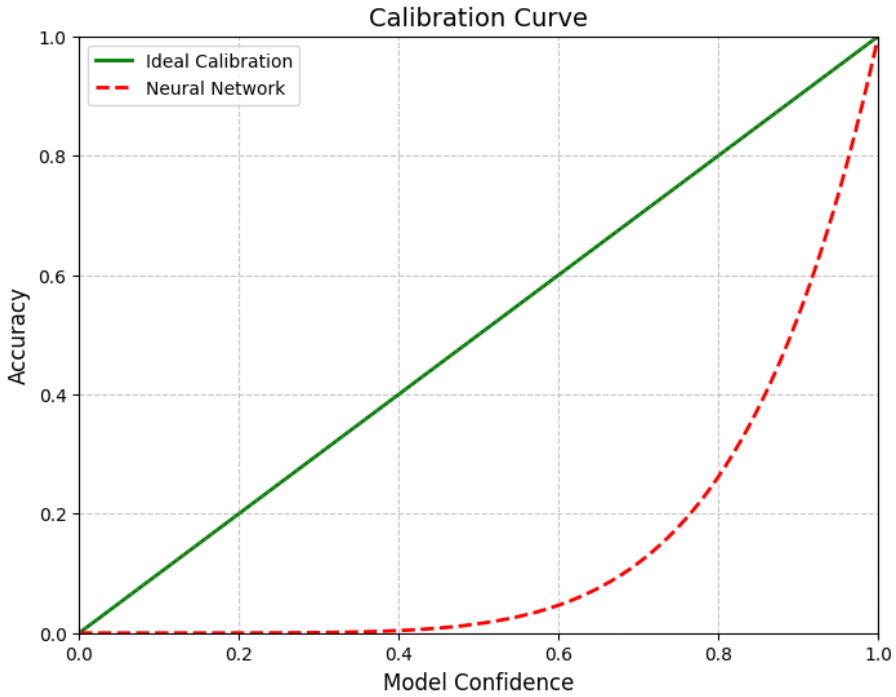
The assumption is that for uncertain tasks, the model would output probabilities in a specific way. However, in practice, when faced with out-of-distribution samples, OOD samples, (e.g., Gaussian noise in an image classification task), the model often exhibits overconfidence in its predictions. This contradicts our initial hypothesis and undermines the reliability of uncertainty estimation.

Illustrations:



We expect a uniform distribution of probabilities across classes for ambiguous inputs. However, in reality, one class dominates, particularly for OOD data. The model consistently exhibits high confidence in its predictions, which invalidates our initial hypothesis.

Calibration curves, as used in traditional machine learning, are challenging to apply here. Unlike binary classification, multi-class tasks require more sophisticated calibration methods, anyway let's illustrate calibration curves, for ideal model and neural network.



Large models tend to suffer from poor calibration: although they achieve higher accuracy rates, their confidence estimates are often misaligned. Smaller models, on the contrary, show better calibration but lower overall accuracy.

We'll discuss two ideas on how to solve this:

1.1 Introducing Temperature τ into Softmax

The greater the difference between logits, the more "peaked" the softmax probabilities become, leading to overconfidence. To mitigate this, we can artificially reduce the logit differences by introducing a temperature parameter τ:

$$p_i = \frac{\exp(x_i/\tau)}{\sum_{k=1}^{c} \exp(x_i/\tau)}$$

When $\tau \to 0$, The probability distribution converges to a delta function at the maximum logit:

$$\tau \to 0 : p \to \delta(\text{argmax}(x)) = (0_1, 0_2, ..., 1_i, ...0_C),$$

where i is the index of the largest logit. This results in extreme confidence in the predicted class.

When $\tau \to \infty$, The probability distribution approaches a uniform distribution across all classes:

$$\tau \to \infty : p \to U(\{1, ..., C\}),$$

meaning all classes receive equal probability, reflecting maximum uncertainty.

In practice, $\tau > 1$ is used.

1.1.1 How to Choose τ?

Can we determine τ using the training data? Let us propose a logical approach:

$$\tau^* = \arg\min_{\tau} \left(-\sum_{n=1}^{N} \sum_{k=1}^{C} [y_n = k] \log p_{nk}^{\tau} \right),$$

where p_{nk}^{τ} denotes the temperature-scaled probability for class k of sample n.

If we optimize this objective over the training set, we are essentially minimizing the standard cross-entropy loss. However, if the model already achieves 100% accuracy on the training set (i.e., all predictions are correct), there are no "negative" examples. In this case, minimizing the loss would incentivize the model to maximize the confidence in the true class by reducing τ to near zero, leading to an overly peaked distribution. This suggests that tuning τ on the training set is flawed.

Instead, validation data containing misclassified examples should be used. For misclassified samples, the loss remains high unless τ is adjusted to soften overconfident predictions. This prevents the model from collapsing to $\tau \to 0$.

1.2 Ensemble (Deep Ensemble)

Deep Ensembling differs from bagging in that it does not require bootstrapping. Due to the inherent bias-variance decomposition in neural networks, training models with different initial-

izations and varying batch orders is sufficient to generate diverse models.

By averaging predictions from multiple models, ensembling improves both accuracy and uncertainty estimation. Specifically, the ensemble reduces variance, leading to better overall performance:

$$p_i^{\text{ens}} = \frac{1}{R} \sum_{r=1}^{R} p_i^r \ ,$$

where R is the number of models in the ensemble.

Consider an ensemble of $R = 3$ models. The first model might predict a random class, the second assigns the highest probability to a different class, and the third produces its own distribution. For ambiguous or out-of-distribution samples, averaging these predictions results in a flatter probability distribution, reflecting lower confidence. If all models are trained with the same number of epochs and protocols (e.g., random seed), direct averaging suffices. However, if trained with varying protocols (e.g., different epoch counts), calibration via temperature scaling is essential.

During training, the norm of the final layer's weights tends to grow, leading to overconfidence. This is analogous to unregularized logistic regression, where large weights produce inflated logits. Without sufficient weight decay to constrain the norm, confidence becomes artificially high. Models trained for more epochs exhibit disproportionately higher confidence, dominating the ensemble's output and skewing results.

Ensemble size is not strictly defined—larger ensembles generally improve performance and uncertainty estimation, but returns diminish as models are added. Performance and uncertainty metrics eventually plateau, requiring a balance between computational cost and gains.

Model independence ensures diverse predictions, but sequential training (e.g., with cosine annealing schedules) can reduce computational overhead while maintaining reasonable diversity. Independence guarantees that no single model overly influences the ensemble, even if trained longer or with differing protocols.

This approach balances accuracy, calibration, and practicality, making deep ensembles a robust method for real-world applications.

Suppose we have trained a large, computationally intensive model with high performance and now aim to optimize it for deployment on mobile devices.

2 (Knowlege) Distillation

This task involves two models: a teacher model (large, high-performance) and a student model (smaller, resource-efficient). Let:

- p_{nk}^T - Probability predictions from the teacher model.
- p_{nk}^S - Probability predictions from the student model.

The standard cross-entropy loss for student is:

$$\text{CrossEntropyLoss} = -\sum_{i=1}^{C} [y_n = k] \log p_{nk}^s \ ,$$

where $[y_n = k]$ are hard class labels.

In distillation, we replace the hard labels with the teacher's probabilistic outputs:

$$\text{DistilationLoss} = -\sum_{i=1}^{C} p_{nk}^T \log p_{nk}^s$$

The final loss combines both objectives:

$$\text{Loss}_{\text{final}} = \alpha \ \text{DistilationLoss} + (1 - \alpha) \, \text{CrossEntropyLoss} \ , \ \alpha \in (0, 1)$$

The teacher's probability distribution contains richer class-specific information, enabling the student to learn nuanced patterns. However, this approach alone cannot achieve excellent performance with very small student models due to their limited capacity.

2.1 Why Does This Work?

Temperature scaling is applied to both p_{nk}^T and p_{nk}^S. Typically, the same temperature τ is used for both:

- Higher τ softens the teacher's distribution, emphasizing inter-class relationships.
- The student mimics this softened distribution, improving generalization.

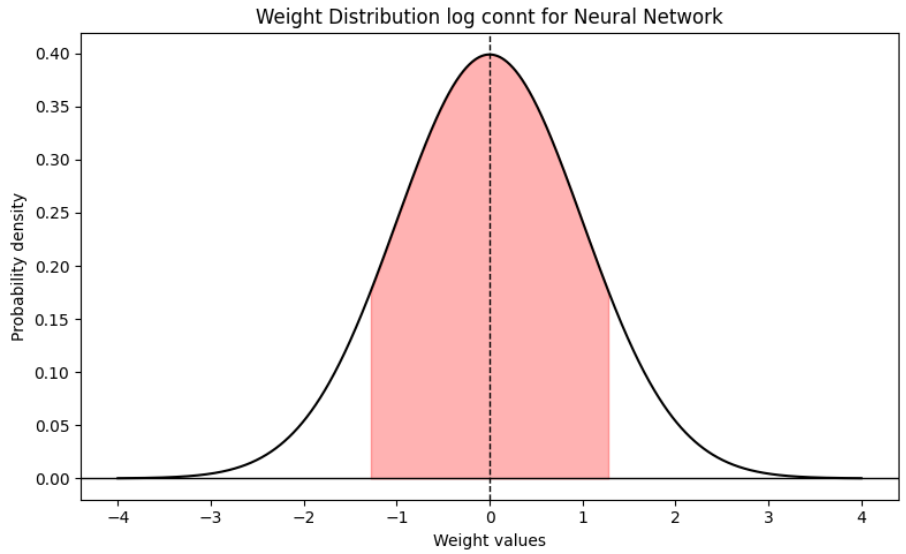
This balances knowledge transfer from the teacher with direct supervision from ground-truth labels, enabling efficient model compression.

3 Pruning

3.1 Unstructured Pruning

In machine learning, pruning originated with decision trees, where an overfitted tree was simplified by optimizing metrics to create a faster, less overfitted version. In deep learning, pruning refers to zeroing out specific weights.

Let‘s look at the plot of weight distribution in a neural network layer.



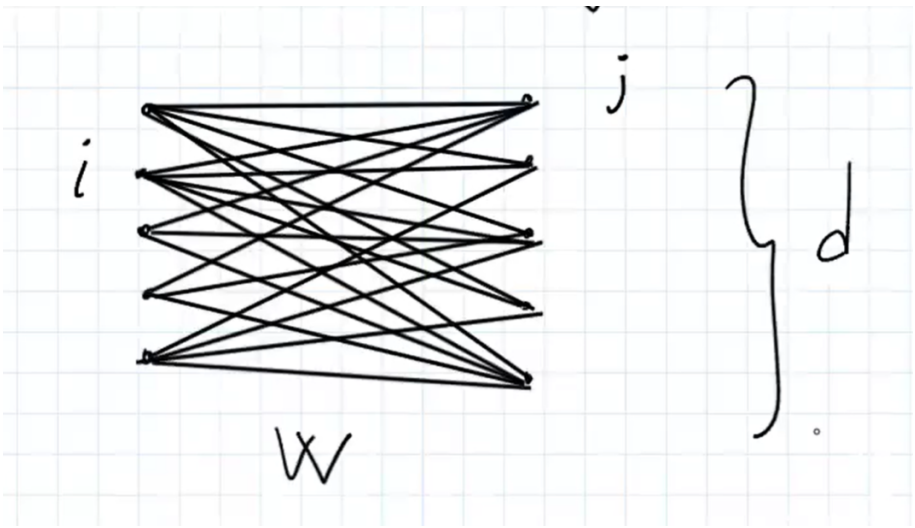
Many weights are near zero, enabling unstructured pruning. Unstructured pruning involves identifying and zeroing out a fraction α of weights with the smallest magnitudes, assuming minimal impact on model performance. This is done independently across layers, as weight magnitude distributions vary by layer.

While implemented in PyTorch, unstructured pruning is inefficient due to floating-point operations and underdeveloped sparse matrix support.

Effective execution requires specialized libraries with boolean masking and hardware optimization

3.2 Structured Pruning

Structured pruning removes entire neurons or channels. For a linear layer:



Structured pruning removes neurons with small weight norms, along with their connections.

A neuron i with output dimension d is pruned if its l_a-norm falls below a threshold:

$$p_i^d = \sqrt[\alpha]{\sum_{j=1}^{d} |w_{ij}|^\alpha}$$

If p_i^d is small, the neuron and all its outgoing connections are removed.

Structured pruning is a more aggressive technique compared to unstructured pruning. Instead of removing individual weights, it eliminates entire neurons along with their outgoing connections, significantly optimizing inference speed. For instance, removing half of the weights might result in only a 1% drop in accuracy. However, achieving extreme sparsity (e.g., pruning 99% of weights) requires a multi-stage approach.

3.3 Multi-Stage Pruning

1. Prune a fraction α of weights.
2. Fine-tune the pruned model to recover accuracy.
3. Repeat until the target sparsity is achieved.

The process begins by pruning a fraction of α the weights, followed by fine-tuning the pruned network to recover performance. This iterative compression minimizes quality degradation compared to applying drastic pruning upfront. While this method reduces model size effectively, it is not a perfect solution—some accuracy loss is inevitable. Nevertheless, in scenarios prioritizing computational efficiency, such trade-offs are often acceptable. The key is to balance compression intensity with retained model performance through careful fine-tuning and incremental pruning steps.

4 Optimization for Linear Layers

Consider a weight matrix of a linear layer:

$$W \in \mathbb{R}^{d_1 \times d_2}$$

We can apply Truncated Singular Value Decomposition (SVD) to factorize it:

$$W = U\Sigma V, U \in \mathbb{R}^{d_1 \times S}, \Sigma \in \mathbb{R}^{S \times S}, V \in \mathbb{R}^{S \times d_2}$$

where S is the number of retained singular values (typically $S \ll \min(d_1, d_2)$).

By applying truncated Singular Value Decomposition (SVD) to the weight matrix $W \in \mathbb{R}^{d_1 \times d_2}$ of a linear layer, we replace the original large matrix with three smaller matrices: $U \in \mathbb{R}^{d_1 \times S}$, $\Sigma \in \mathbb{R}^{S \times S}$ (a diagonal matrix of singular values), and $V \in \mathbb{R}^{S \times d_2}$. This decomposition reduces computational complexity, as matrix multiplications involving U, Σ and V require fewer operations compared to directly computing Wx Specifically, the diagonal matrix Σ scales the singular values, enabling efficient rank reduction.

The SVD can be applied either dynamically during training or as a one-time post-training optimization. While this approach is highly effective for linear layers—especially in transformer architectures with large "inverted bottleneck" layers—it is poorly suited for convolutional layers due to their spatial structure. In transformers, where linear operations dominate, SVD compression significantly reduces computational costs without drastic accuracy loss.

The key advantage lies in replacing a single high-dimensional matrix multiplication with two sequential operations: $V^T x$ followed by $\Sigma\left(V^T x\right)$, and finally $U\left(\Sigma\left(V^T x\right)\right)$. This reduces the complexity from $O\left(d_1 d_2\right)$ to $O\left(d_1 S + S^2 + d_2 S\right)$, where $S \ll \min\left(d_1, d_2\right)$.

However, balancing the compression ratio (S) and model accuracy remains critical, particularly when targeting extreme sparsity.

5 Quantization

Neural networks are typically trained using floating-point numbers (e.g., float32 in PyTorch), with mixed-precision training occasionally employing float16. The core idea of quantization is to convert the model to int8—a seemingly counterintuitive approach due to reduced bitwidth. However, int8 operations are significantly faster, even though floating-point computations rely on specialized FPU hardware. While this risks quality degradation, empirical observations show that most neural network weights cluster near zero, making quantization feasible.

5.1 Quantization of a Fully Connected Layer

Consider a weight matrix $W \in \mathbb{R}^{d_1 \times d_2}$, where each weight $w$is stored as float32. Quantization maps maps w to int8 using:

$$w = S_W\left(w^q - z_w\right) \ ,$$

Where:

- $S_W \in$ float32, a scaling factor,
- $z_W \in$ int8, is a zero-point offset,
- $w^q \in$ int8, is the quantized weight.

The quantization process involves:

1. Scaling and Rounding:

$$w^q = \text{clip}\left(\text{round}\left(\frac{w}{S_W}\right) + z_W, -128, 127\right) \ ,$$

   where clipping ensures values stay within the int8 range.

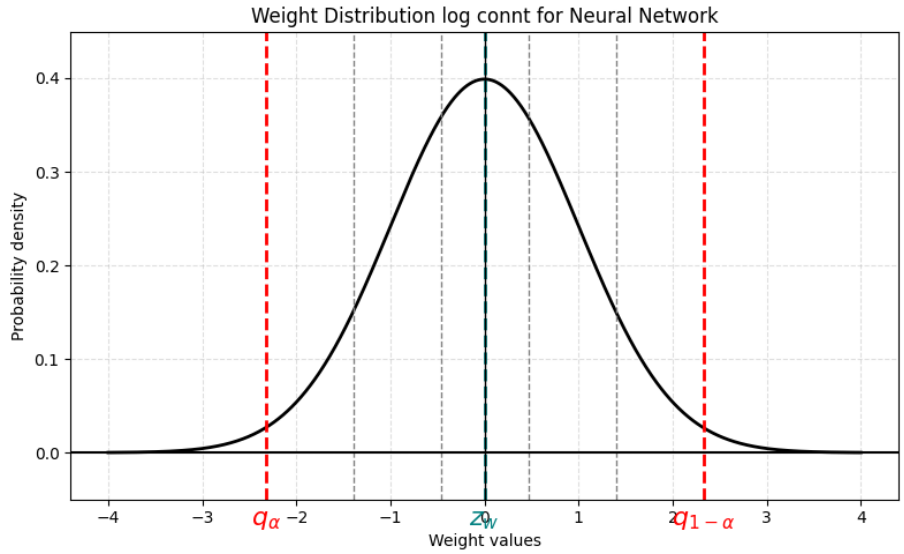
2. Parameter Calibration:

The scaling factor S_W and zero-point z_W are derived from the weight distribution. By selecting quantiles q_a and q_{1-a}, that capture most weights, we map:

$$q_a \to 128, q_{1-a} \to 127$$

The zero-point is then:

$$z_W = \text{round}\left(\frac{q_{1-a} + q_a}{2}\right)$$



Let's see how the quantized pass works for a matrix of linear weights

5.2 Quantized Matrix Multiplication

For a linear layer with input $X \in \mathbb{R}^{B\times d_1}$, output $Y \in \mathbb{R}^{B\times d_2}$, and quantized parameters S_x, z_x (input) and S_W, z_W(weights), the computational becomes

$$Y = XW, y_{ij} = \sum_{k=1}^{d_1} x_{ik} w_{kj}$$

$$y_{ij} = \sum_{k=1}^{d_1} s_x \left(x_{ik}^q - z_x\right) \times s_w \left(w_{kj}^q - z_w\right) = s_x s_w \left[\sum_{k=1}^{d_1} x_{ik}^q w_{kj}^q - z_x \sum_{k=1}^{d_1} w_{kj}^q - z_W \sum_{k=1}^{d_1} x_{ik^q} + d1 z_x z_w\right]$$

$$= S_Y \left(y_{ij}^q - z_Y\right)$$

Matrix multiplication will only take place at this stage $\sum_{k=1}^{d_1} x_{ik}^q w_{kj}^q$, but they are in int8. We also need quantization parameters for the input (S_x, Z_x) and output (S_W, Z_W). These parameters are calculated by passing over the training sample, let's calculate typical values received magnitude distribution for each.

In PyTorch's quantization implementation, GPU support is unavailable, so significant acceleration is not achieved. However, since mobile devices also lack GPUs, training is performed on the CPU

6 Additional Methods

Active research is underway to develop advanced optimization techniques, including decomposition methods and approaches inspired by works like GPTQ. For transformer models, recent studies propose quantization strategies that intelligently select which parameters to quantize or prune. This is achieved by minimizing the Frobenius norm difference between the quantized and original (non-quantized) layers.

The key innovation lies not only in choosing optimal weights for quantization but also in dynamically adjusting non-quantized weights (stored in float32) to compensate for quantization errors. By iteratively refining these weights, the method mitigates accuracy degradation while maintaining computational efficiency.

Key Features:

- Targeted Quantization: Prioritizes layers or parameters where quantization introduces minimal error.
- Error Correction: Non-quantized weights are fine-tuned to offset distortions caused by quantization.
- Adaptive Workflow: Combines pruning and quantization into a unified framework, optimizing both model size and inference speed.

This approach bridges the gap between aggressive compression and model performance, making it particularly effective for deploying large transformers on resource-constrained devices. LINK